```
/* درب الحاج — app logic. Plain JavaScript, no build step: works as-is on GitHub
'use strict';

/* ==================== group setup (edit here if needed) ====================
const CONFIG = {
  group: 'GRP-014',
  groupName: 'مجموعة مسار الأمن',
  supervisor: { ref: 'SUP-014', name: 'وتين الشهيب' },
  pilgrims: ['HJ-2648-971', 'HJ-2648-972', 'HJ-2648-973', 'HJ-2648-974', 'HJ-2648
  // live messages between phones go through the free ntfy.sh relay; the channel
  relay: 'https://ntfy.sh',
  topic: 'darb-alhajj-p10p3nwq2832sodbwnus',
  stages: ['الحرم', 'منى', 'عرفات', 'مزدلفة'],

  // التواصل الرسمي للفريق (05 xxxxxxxx — الرقم يُكتب هنا بصيغة) دم للاتصال والواتساب
  contact: { phone: '', email: 'darbalhaj27@gmail.com' },

  team: [
    { name: 'وتين الشهيب', role: 'قائدة الفريق والتخطيط', did: 'متابعة عمل الفريق
    { name: 'رغد بخاري', role: 'تحليل وتصميم', did: 'لة وتصميم الواجهات والمخططات
    { name: 'خزامى الغامدي', role: 'تحليل وتصميم', did: 'تصميم الواجهات والمخططات
    { name: 'لينه العدناني', role: 'تطوير واختبار', did: 'طبيق وتجربته على الأجهزة
    { name: 'سفانة الحربي', role: 'تطوير واختبار', did: 'تطبيق وتجربته على الأجهزة
    { name: 'ملك الأسمري', role: 'تحليل وتصميم', did: 'ة وتصميم الواجهات والمخططات
  ],

  // جدول المجموعة: stage = (تُظهر «الآن» على أول محطة لها) حلة التي يختارها المشرف
  schedule: [
    { stage: 'الحرم', day: '8 ذو الحجة', tag: 'يوم التروية', time: '7:00 ص', name
    { stage: 'منى', day: '8 ذو الحجة', time: '10:00 ص', name: '12 نى — المخيم رقم
    { stage: 'عرفات', day: '9 ذو الحجة', tag: 'يوم عرفة', time: '6:00 ص', name: '
    { stage: 'مزدلفة', day: '9 ذو الحجة', time: 'بعد الغروب', name: 'مزدلفة', not
    { day: '10 ذو الحجة', tag: 'يوم النحر', time: 'بعد الفجر', name: 'ى — الجمرات
    { day: '10 ذو الحجة', time: 'العصر', name: 'المسجد الحرام', note: 'اضة والسعي
    { day: '11 – 13 ذو الحجة', tag: 'أيام التشريق', time: 'يوميًا', name: 'مِنى', n
    { day: '13 ذو الحجة', time: 'قبل المغادرة', name: 'المسجد الحرام', note: 'داع
  ],

  emergency: [
    { n: '911', t: 'الطوارئ الموحد' },
    { n: '997', t: 'الهلال الأحمر' },
    { n: '1966', t: 'وزارة الحج والعمرة' },
    { n: '937', t: 'وزارة الصحة' },
```

```
  ],
};
if (window.__RELAY__) CONFIG.relay = window.__RELAY__;            // automated te
CONFIG.alertTopic = CONFIG.topic + '-alerts';                     // only report

/* languages (i18n.js): T('نص عربي') returns the text in the chosen language. Dat
   (report types, stages, preset notices) always stays in Arabic and is translate
if (!window.I18N) window.I18N = { lang: 'ar', dir: 'rtl', langs: [], T: (s, v) =>
const T = window.I18N.T;

/* ==================== small helpers ==================== */
const $ = (s, r = document) => r.querySelector(s);
const $$ = (s, r = document) => Array.from(r.querySelectorAll(s));
const store = {
  get(k, d) { try { const v = localStorage.getItem('darb.' + k); return v == null
  set(k, v) { try { localStorage.setItem('darb.' + k, JSON.stringify(v)); } catch
  del(k) { try { localStorage.removeItem('darb.' + k); } catch (e) { /* ignore */
};
const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2, 8
const esc = (s) => String(s == null ? '' : s).replace(/[&<>"']/g, (c) => ({ '&':
const bdi = (s) => `<bdi class="ref">${esc(s)}</bdi>`;
function normRef(s) {
  s = String(s || '')
    .replace(/[٩-٠]/g, (d) => String('٠١٢٣٤٥٦٧٨٩'.indexOf(d)))
    .replace(/[٠-٩]/g, (d) => String('٠١٢٣٤٥٦٧٨٩'.indexOf(d)))
    .toUpperCase().replace(/[--_\s]/g, (c) => (c.trim() ? '-' : ''));
  const m = s.match(/HJ-?\d{4}-?\d{3}|SUP-?\d{3}/);
  if (!m) return s;
  const r = m[0];
  if (r.startsWith('SUP')) return 'SUP-' + r.replace(/\D/g, '');
  const d = r.replace(/\D/g, '');
  return 'HJ-' + d.slice(0, 4) + '-' + d.slice(4);
}
function meters(a, b) {
  if (!a || !b || a.lat == null || b.lat == null) return null;
  const R = 6371000, k = Math.PI / 180;
  const dLat = (b.lat - a.lat) * k, dLon = (b.lon - a.lon) * k;
  const s = Math.sin(dLat / 2) ** 2 + Math.cos(a.lat * k) * Math.cos(b.lat * k) *
  return 2 * R * Math.asin(Math.sqrt(s));
}
const fmtDist = (m) => (m == null ? '—' : m < 1000 ? T('{n} م', { n: Math.round(m
function ago(ts) {
  const s = Math.max(0, (Date.now() - ts) / 1000);
  const m = Math.round(s / 60), h = Math.round(m / 60);
  if (I18N.lang !== 'ar') {
    const rtf = I18N.rtf();
```

```
    if (rtf) return s < 45 ? rtf.format(0, 'second') : m < 60 ? rtf.format(-m, 'm
    return s < 45 ? T('الآن') : m < 60 ? T('قبل {n} د', { n: m }) : T('قبل {n} س',
  }
  if (s < 45) return 'الآن';
  if (m < 60) return m === 1 ? 'قبل دقيقة' : m === 2 ? 'قبل' : 'قبل دقيقتين' + m
  return h === 1 ? 'قبل' : 'قبل ساعة' + h + ' س';
}
const clock = (t) => { const d = new Date(t); return String(d.getHours()).padStar
const haptic = (p = 40) => { try { navigator.vibrate && navigator.vibrate(p); } c
const loaded = {};
function loadScript(src) {
  return loaded[src] || (loaded[src] = new Promise((res, rej) => {
    const s = document.createElement('script'); s.src = src; s.onload = res; s.on
    document.head.appendChild(s);
  }));
}
function loadCSS(href) {
  if (loaded[href]) return; loaded[href] = true;
  const l = document.createElement('link'); l.rel = 'stylesheet'; l.href = href;
}
let toastTimer;
function toast(msg, ms = 2800) {
  const t = $('#toast'); t.textContent = msg; t.classList.add('show');
  clearTimeout(toastTimer); toastTimer = setTimeout(() => t.classList.remove('sho
}
function shake(sel) { const i = $(sel); if (!i) return; i.classList.remove('shake

/* ==================== sound (Web Audio, unlocked by the first tap) ==========
const Sound = {
  ctx: null,
  unlock() {
    try {
      if (!this.ctx) this.ctx = new (window.AudioContext || window.webkitAudioCon
      if (this.ctx.state === 'suspended') this.ctx.resume();
    } catch (e) { /* no audio */ }
  },
  beep(seq, vol = 0.32) {
    const c = this.ctx; if (!c) return;
    let t = c.currentTime + 0.02;
    seq.forEach(([f, d]) => {
      const o = c.createOscillator(), g = c.createGain();
      o.type = 'sine'; o.frequency.value = f;
      g.gain.setValueAtTime(0.0001, t);
      g.gain.exponentialRampToValueAtTime(vol, t + 0.02);
      g.gain.exponentialRampToValueAtTime(0.0001, t + d);
      o.connect(g).connect(c.destination); o.start(t); o.stop(t + d + 0.03);
```

```
      t += d + 0.05;
    });
  },
  alarm() { this.beep([[880, 0.16], [660, 0.16], [880, 0.16], [660, 0.16], [988,
  chime() { this.beep([[784, 0.13], [1047, 0.24]], 0.22); },
};
document.addEventListener('pointerdown', () => Sound.unlock(), { passive: true })

/* ==================== session ==================== */
const Session = {
  role: null, ref: null,
  load() { const s = store.get('session', null); if (s && s.role && s.ref) { this
  save() { store.set('session', { role: this.role, ref: this.ref }); },
  clear() { this.role = this.ref = null; store.del('session'); },
};

/* ==================== shared state, rebuilt from events ====================
const S = {
  events: new Map(),
  members: {},              // ref -> {lat, lon, acc, t}
  reports: {},              // rid -> {rid, ref, type, t, lat, lon, acc, trail:[]}
  status: {},               // rid -> {seen, ack, resolve, cancel} (times)
  sup: { lat: null, lon: null, acc: null, t: 0, phone: null },
  stage: null,              // {stage, t}
  notices: [],              // {id, text, t}
  clearedAt: 0,
  keepOpenAfter: 0,           // after a supervisor login: open reports newer than
  answered: new Set(),
};
const isSupRef = (r) => r === CONFIG.supervisor.ref;
const evTime = (ev) => ev._st || ev.t;
const fresh = (ev, ms) => Date.now() - evTime(ev) < ms;

function ingest(ev, src) {
  if (!ev || !ev.id || S.events.has(ev.id)) return;
  S.events.set(ev.id, ev);
  if (src !== 'cache') cacheEvent(ev);
  const fromSup = isSupRef(ev.from);
  const live = src === 'remote' || src === 'local';
  const hasLoc = ev.lat != null && ev.lon != null;

  if (hasLoc || ev.k === 'hello') {
    if (fromSup) {
      if (hasLoc && ev.t >= S.sup.t) Object.assign(S.sup, { lat: ev.lat, lon: ev.
    } else if (CONFIG.pilgrims.includes(ev.from)) {
      const m = S.members[ev.from] || (S.members[ev.from] = { t: 0 });
```

```
    if (ev.t >= m.t) { m.t = ev.t; if (hasLoc) Object.assign(m, { lat: ev.lat,
  }
}
if (fromSup && ev.phone !== undefined && ev.t >= (S.sup.phoneT || 0)) { S.sup.p

switch (ev.k) {
  case 'hello':
    if (live && src === 'remote' && Session.role === 'super' && !fromSup && fre
    break;
  case 'report':
    if (!CONFIG.pilgrims.includes(ev.from) || !survivesClear(ev.t)) break;
    S.reports[ev.rid] = { rid: ev.rid, ref: ev.from, type: ev.type || 'بلاغ', t:
    break;
  case 'found':
    if (!CONFIG.pilgrims.includes(ev.ref) || !survivesClear(ev.t)) break;
    S.reports[ev.rid] = { rid: ev.rid, ref: ev.ref, type: 'يه – بلاغ من شخص وجده
    break;
  case 'loc':
    if (ev.rid && S.reports[ev.rid] && ev.from === S.reports[ev.rid].ref && has
      const r = S.reports[ev.rid];
      Object.assign(r, { lat: ev.lat, lon: ev.lon, acc: ev.acc });
      r.trail.push([ev.lat, ev.lon]);
    }
    break;
  case 'seen': case 'ack': case 'resolve':
    if (!fromSup) break;
    (S.status[ev.rid] || (S.status[ev.rid] = {}))[ev.k] = ev.t;
    break;
  case 'cancel': {
    const r = S.reports[ev.rid];
    if (!r || r.ref !== ev.from) break;
    (S.status[ev.rid] || (S.status[ev.rid] = {})).cancel = ev.t;
    if (live && src === 'remote' && Session.role === 'super' && fresh(ev, 12000
    break;
  }
  case 'ping':
    if (fromSup) answerPing(ev, src);
    break;
  case 'stage':
    if (!fromSup || (S.stage && S.stage.t >= ev.t)) break;
    S.stage = { stage: ev.stage, t: ev.t };
    if (src === 'remote' && Session.role === 'pilgrim' && fresh(ev, 120000)) {
    break;
  case 'notice':
    if (!fromSup || ev.t < S.clearedAt) break;
    S.notices.push({ id: ev.id, text: String(ev.text || '').slice(0, 140), t: e
```

```
      if (src === 'remote' && Session.role === 'pilgrim' && fresh(ev, 120000)) {
        MT.ready(String(ev.text || '').slice(0, 140)).then((txt) => toast(T('📣 a
        Sound.chime(); haptic([120, 80, 120]);
      }
      break;
    case 'clear': {
      const cut = Math.min(ev.before || ev.t, ev.t);
      if (!fromSup || cut <= S.clearedAt) break;
      S.clearedAt = cut;
      S.keepOpenAfter = ev.keepOpenAfter || 0;
      Object.keys(S.reports).forEach((rid) => {
        const r = S.reports[rid];
        if (r.t >= cut) return;
        if (S.keepOpenAfter && r.t >= S.keepOpenAfter) r.preClear = true;   // sh
        else delete S.reports[rid];
      });
      S.notices = S.notices.filter((n) => n.t >= cut);
      break;
    }
    default:
  }
  scheduleRender();
}

function survivesClear(t) { return t >= S.clearedAt || (S.keepOpenAfter && t >= S
const visibleReport = (r) => !r.preClear || isOpen(r);       // closed reports fr
function reportStatus(rid) {
  const st = S.status[rid] || {};
  if (st.cancel) return 'cancel';
  if (st.resolve) return 'resolve';
  if (st.ack) return 'ack';
  if (st.seen) return 'seen';
  return 'sent';
}
const isOpen = (r) => r && Date.now() - r.t < 6 * 3600e3 && !['cancel', 'resolve'
const openReports = () => Object.values(S.reports).filter(isOpen).sort((a, b) =>
const openReportOf = (ref) => openReports().find((r) => r.ref === ref) || null;
const isOnline = (m) => !!m && m.t >= S.clearedAt && Date.now() - m.t < 15 * 6000
const thisSession = (m) => (m && m.t >= S.clearedAt ? m : null);   // presence fr
function myPos() { return Geo.last; }
function supPos() { return Session.role === 'super' ? (Geo.last || (S.sup.lat !=

/* local cache of events: keeps the same-phone demo working offline, and makes re
let cacheTimer = null;
const cacheBuf = [];
function cacheEvent(ev) {
```

```
  cacheBuf.push(ev);
  clearTimeout(cacheTimer);
  cacheTimer = setTimeout(() => {
    const all = store.get('events', []).concat(cacheBuf.splice(0));
    store.set('events', all.filter((e) => Date.now() - e.t < 12 * 3600e3).slice(-
  }, 300);
}
function loadCache() { store.get('events', []).filter((ev) => Date.now() - ev.t <

/* ==================== relay: ntfy.sh (publish = POST, listen = Server-Sent Eve
const Relay = {
  es: null, online: false, outbox: store.get('outbox', []), backoff: 4000, rt: 0,
  url(p = '') { return CONFIG.relay.replace(/\/$/, '') + '/' + CONFIG.topic + p;
  publish(fields) {
    const ev = Object.assign({ v: 1, id: uid(), t: Date.now(), g: CONFIG.group, f
    ingest(ev, 'local');
    this.outbox.push(ev);
    store.set('outbox', this.outbox.slice(-80));
    this.flush();
    return ev;
  },
  async flush() {
    if (this.flushing || !this.outbox.length) return;
    this.flushing = true;
    try {
      while (this.outbox.length) {
        const ev = this.outbox[0];
        if (Date.now() - ev.t > 6 * 3600e3) { this.outbox.shift(); continue; }
        const r = await fetch(this.url(), { method: 'POST', body: JSON.stringify(
        if (r.status === 429) throw Object.assign(new Error('rate'), { wait: 6000
        if (!r.ok) throw new Error('HTTP ' + r.status);
        this.outbox.shift();
        store.set('outbox', this.outbox.slice(-80));
        this.setOnline(true);
      }
    } catch (e) {
      this.setOnline(false);
      setTimeout(() => this.flush(), e.wait || 8000);
    } finally {
      this.flushing = false;
    }
  },
  connect() {
    if (this.es) return;
    clearTimeout(this.rt);
    this.lastAttempt = Date.now();
```

```
    const last = store.get('lastMsg', null);
    const since = last && Date.now() - last.at < 11 * 3600e3 ? last.id : '12h';
    let es;
    try { es = new EventSource(this.url('/sse?since=' + encodeURIComponent(since)
    this.es = es;
    es.onopen = () => { this.backoff = 4000; this.setOnline(true); this.flush();
    es.onmessage = (e) => {
      let m; try { m = JSON.parse(e.data); } catch (_) { return; }
      if (m.event !== 'message' || !m.message) return;
      store.set('lastMsg', { id: m.id, at: Date.now() });
      let ev; try { ev = JSON.parse(m.message); } catch (_) { return; }
      if (!ev || ev.v !== 1 || ev.g !== CONFIG.group) return;
      ev._st = m.time ? m.time * 1000 : 0;
      ingest(ev, 'remote');
    };
    es.onerror = () => { this.disconnect(true); this.retry(); };
  },
  retry() {
    clearTimeout(this.rt);
    this.rt = setTimeout(() => this.connect(), this.backoff);
    this.backoff = Math.min(60000, this.backoff * 2);
  },
  disconnect(keepRetry) {
    if (this.es) { this.es.close(); this.es = null; }
    if (!keepRetry) clearTimeout(this.rt);
    this.setOnline(false);
  },
  refresh() {                       // after the phone wakes up: reopen the stream
    if (!Session.role || Date.now() - this.lastAttempt < 3000) return;
    this.disconnect(); this.backoff = 4000; this.connect();
  },
  setOnline(on) { if (this.online === on) return; this.online = on; renderConn();
};

/* ==================== location ==================== */
const Geo = {
  last: null, watchId: null, subs: new Set(), denied: false,
  get(maxAge = 15000) {
    return new Promise((res, rej) => {
      if (!navigator.geolocation) return rej(new Error('nogeo'));
      navigator.geolocation.getCurrentPosition((p) => { this.update(p); res(this.
        (e) => { if (e && e.code === 1) this.denied = true; scheduleRender(); rej
        { enableHighAccuracy: true, timeout: 15000, maximumAge: maxAge });
    });
  },
  update(p) {
```

```
    this.last = { lat: +p.coords.latitude.toFixed(6), lon: +p.coords.longitude.to
    this.lastAt = Date.now();
    this.denied = false;
    this.subs.forEach((f) => f(this.last));
    scheduleRender();
  },
  watch(f) {
    this.subs.add(f);
    if (this.watchId == null && navigator.geolocation) {
      this.watchId = navigator.geolocation.watchPosition((p) => this.update(p),
        (e) => { if (e && e.code === 1) { this.denied = true; scheduleRender(); }
        { enableHighAccuracy: true, maximumAge: 5000, timeout: 20000 });
    }
  },
  unwatch(f) {
    this.subs.delete(f);
    if (!this.subs.size && this.watchId != null) { navigator.geolocation.clearWat
  },
};

/* live location while a report is open (pilgrim) or while the supervisor is on t
function throttledSender(extra) {
  let last = null, n = 0, pending = null, timer = 0;
  const send = (p) => {
    if (++n > 150) return;
    last = Object.assign({ t: Date.now() }, p); pending = null;
    Relay.publish(Object.assign({ k: 'loc' }, extra(), p));
  };
  const f = function (p) {
    const now = Date.now();
    if (!last) { send(p); return; }
    const moved = meters(last, p);
    if (now - last.t > 90000 || (moved > 8 && now - last.t > 20000)) { clearTimeo
    if (moved > 8) {                                    // too soon: send the newe
      pending = p;
      clearTimeout(timer);
      timer = setTimeout(() => { if (pending) send(pending); }, 20000 - (now - la
    }
  };
  f.cancel = () => clearTimeout(timer);
  return f;
}
let liveShare = null;
function startLiveShare(rid) {
  stopLiveShare();
  const f = throttledSender(() => ({ rid }));
```

```
  liveShare = { rid, f };
  Geo.watch(f);
}
function stopLiveShare() { if (liveShare) { liveShare.f.cancel(); Geo.unwatch(liv
let supShare = null;
function syncSupShare() {
  const need = Session.role === 'super' && openReports().some((r) => reportStatus
  if (need && !supShare) { supShare = throttledSender(() => ({})); Geo.watch(supS
  if (!need && supShare) { supShare.cancel(); Geo.unwatch(supShare); supShare = n
}
function onSupFix() { /* keeps Geo.last fresh on the supervisor phone for distanc

function answerPing(ev, src) {
  if (Session.role !== 'pilgrim' || src === 'cache' || src === 'local' || S.answe
  S.answered.add(ev.id);
  setTimeout(() => {
    Geo.get(10000).then((p) => Relay.publish(Object.assign({ k: 'loc' }, p)))
      .catch(() => Relay.publish({ k: 'hello', role: 'pilgrim' }));
  }, 400 + Math.random() * 2000);
}

/* ==================== QR scanner (camera) ==================== */
const Scanner = {
  stream: null, timer: 0, box: null, cb: null, detector: null, canvas: null,
  async start(box, cb) {
    this.stop();
    this.box = box; this.cb = cb;
    if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) {
      toast(T('الكاميرا غير متاحة هنا — اكتب الرقم المرجعي بالأسفل')); return fals
    }
    try {
      this.stream = await navigator.mediaDevices.getUserMedia({ audio: false, vid
    } catch (e) {
      toast(T(e && e.name === 'NotAllowedError' ? 'ر ا — اكتب الرقم المرجعي بالأسفل'
      return false;
    }
    if (this.box !== box) { this.stream.getTracks().forEach((t) => t.stop()); ret
    const v = box.querySelector('video');
    v.setAttribute('playsinline', ''); v.muted = true; v.srcObject = this.stream;
    try { await v.play(); } catch (e) { /* iOS may need a tap; the stream still f
    box.classList.add('live');
    if (!this.detector && 'BarcodeDetector' in window) {
      try { const f = await window.BarcodeDetector.getSupportedFormats(); if (f.i
    }
    if (!this.detector) { try { await loadScript('jsQR.js'); } catch (e) { toast(
    this.loop(v);
```

```
    return true;
  },
  loop(v) {
    const tick = async () => {
      if (!this.stream) return;
      let text = null;
      if (v.readyState >= 2 && v.videoWidth) {
        try {
          if (this.detector) {
            const r = await this.detector.detect(v);
            if (r && r[0]) text = r[0].rawValue;
          } else if (window.jsQR) {
            const w = 640, h = Math.round((w * v.videoHeight) / v.videoWidth);
            const c = this.canvas || (this.canvas = document.createElement('canva
            c.width = w; c.height = h;
            const ctx = c.getContext('2d', { willReadFrequently: true });
            ctx.drawImage(v, 0, 0, w, h);
            const r = window.jsQR(ctx.getImageData(0, 0, w, h).data, w, h, { inve
            if (r && r.data) text = r.data;
          }
        } catch (e) { /* keep scanning */ }
      }
      if (text) {
        const box = this.box, cb = this.cb;
        box.classList.add('hit'); setTimeout(() => box.classList.remove('hit'), 7
        this.stop(); haptic([40, 30, 40]);
        if (cb) cb(text);
        return;
      }
      this.timer = setTimeout(tick, 120);
    };
    tick();
  },
  stop() {
    clearTimeout(this.timer);
    if (this.stream) { this.stream.getTracks().forEach((t) => t.stop()); this.str
    if (this.box) { const v = this.box.querySelector('video'); if (v) v.srcObject
  },
};
function scanFor(target) {
  const box = target === 'pilgrim' ? $('#pScanner') : $('#sScanner');
  Scanner.start(box, (code) => (target === 'pilgrim' ? pilgrimLogin(code) : super
}

/* ==================== navigation ==================== */
const PAGE_NAMES = { home: 'الرئيسية', places: 'المواقع والمواعيد', scan: 'الرمز
```

```
let current = 'home';
function go(id, fromPop) {
  if (id === current) return;
  if (!fromPop) {
    if (id !== 'home' && current === 'home') history.pushState({ s: id }, '');
    else if (id === 'home' && history.state && history.state.s) { history.back();
  }
  Scanner.stop();
  const from = $('#' + current), to = $('#' + id);
  from.classList.remove('active'); from.classList.add('out');
  setTimeout(() => from.classList.remove('out'), 600);
  to.classList.add('active'); to.scrollTop = 0;
  current = id;
  $$('.navbtn').forEach((b) => b.classList.toggle('active', b.dataset.go === id))
  $('#pageName').textContent = T(PAGE_NAMES[id] || '');
  if (id === 'places') onPlacesOpen();
  if (id === 'contact') renderContact();
  if (id === 'home') runRoute();
  if (id === 'super' && Session.role === 'super') { initMap(); keepAwake(true); }
  updateK3dActive();
  scheduleRender();
}
window.addEventListener('popstate', () => {
  if (k3dFullOpen) { closeK3dFull(true); return; }
  if ($('.sheet.show')) closeSheets();
  if (current !== 'home') go('home', true);
});
function show(id) { $('#veil').classList.add('show'); $('#' + id).classList.add('
function closeSheets() { $$('.sheet').forEach((s) => s.classList.remove('show'));
function onLoginPill() {
  if (Session.role === 'pilgrim') go('pilgrim');
  else if (Session.role === 'super') go('super');
  else show('loginSheet');
}
function updatePill() {
  const p = $('#loginPill');
  if (Session.role === 'pilgrim') p.innerHTML = `<span class="pw">${esc(T('حاج'))
  else if (Session.role === 'super') p.innerHTML = esc(T('المشرفة · لوحة المتابعة
  else p.textContent = T('تسجيل الدخول');
}

/* ==================== pilgrim ==================== */
function pilgrimLogin(raw) {
  const ref = normRef(raw);
  if (!CONFIG.pilgrims.includes(ref)) {
    shake('#pRef');
```

```
    toast(T(isSupRef(ref) ? 'الرقم المرجعي غير موجود في المجموعة' : 'دخول كمشرف»'
    haptic([100, 50, 100]);
    return;
  }
  if (Session.role === 'super') leaveSupervisor();
  Session.role = 'pilgrim'; Session.ref = ref; Session.save();
  Scanner.stop();
  $('#pRef').value = '';
  updatePill();
  Relay.connect();
  pilgrimFirstRender = true;
  Object.values(S.reports).forEach((r) => {             // a fresh login closes m
    if (r.ref === ref && !r.found && isOpen(r) && Date.now() - r.t > 10 * 60000)
  });
  scheduleRender();
  toast(T('تم ربطك بالمشرفة 🤍 أهلاً بك {name}', { name: CONFIG.supervisor.name }))
  Geo.get().then((p) => Relay.publish(Object.assign({ k: 'hello', role: 'pilgrim'
    .catch(() => Relay.publish({ k: 'hello', role: 'pilgrim' }));
}
const myOpenReport = () => (Session.role === 'pilgrim' ? openReports().find((r) =
function myLatestReport() {
  return Object.values(S.reports).filter((r) => r.ref === Session.ref && !r.found
}
let rType = 'مفقود / تائه', rFix = null, locRow = ['loading', null];
function openReport() {
  if (myOpenReport()) { toast(T('عندك بلاغ مفتوح — المشرفة تتابعه الآن')); return;
  renderReportFor();
  rFix = null; setLocRow('loading');
  show('reportSheet');
  Geo.get(10000).then((p) => { rFix = p; setLocRow('ok', p); }).catch(() => setLo
}
function renderReportFor() {
  $('#rFor').innerHTML = T('من الرقم المرجعي {ref} إلى المشرفة {name}', { ref: bd
}
function setLocRow(state, p) {
  locRow = [state, p];
  const row = {
    loading: ['جارٍ تحديد موقعك', 'موقعك الحالي…'],
    ok: ['تم تحديد موقعك {n} ✓', 'بدقة'], ['م — يوصل للمشرفة مع البلاغ ويتحدث مباشرة
    denied: ['فعّل خدمة الموقع للمتصفح من الإعدادات — البلاغ يوصل بدون موقع', 'مسموح'],
    fail: ['سيوصل البلاغ الآن، ويُرسل الموقع أول ما يتوفر', 'تعذّر تحديد الموقع'],
  }[state];
  $('#rLoc div').innerHTML = `<b>${esc(T(row[0]))}</b><small>${esc(T(row[1], { n:
}
$$('#rTypes .chip').forEach((c) => c.addEventListener('click', () => {
  $$('#rTypes .chip').forEach((x) => x.classList.remove('on')); c.classList.add('
```

```
}));
function sendReport() {
  if (myOpenReport()) { closeSheets(); return; }
  const rid = uid();
  const p = rFix || Geo.last || {};
  Relay.publish(Object.assign({ k: 'report', rid, type: rType }, p.lat != null ?
  publishAlert('report', Session.ref, rType, p);
  closeSheets();
  showSuccess(T('تم إرسال البلاغ'), T('يوصل التنبيه لجوال المشرفة الآن مع موقعك'));
  haptic([60, 40, 60]);
  startLiveShare(rid);
}
function cancelReport() {
  const r = myOpenReport(); if (!r) return;
  if (!confirm(T('إلغاء البلاغ؟'))) return;
  Relay.publish({ k: 'cancel', rid: r.rid });
  stopLiveShare();
}
function shareMyLocation() {
  toast(T('جارٍ تحديد موقعك…'));
  Geo.get(5000).then((p) => { Relay.publish(Object.assign({ k: 'loc' }, p)); toas
    .catch(() => toast(T(Geo.denied ? 'تعذّر تحديد الموقع، حاول مرة ثانية' : 'ادات
}
function callSupervisor() {
  const n = S.sup.phone || CONFIG.contact.phone;
  if (n) location.href = 'tel:' + n;
  else toast(T('رقم التواصل غير مضاف بعد — ارفع بلاغ وبيوصل للمشرفة فورًا'), 3800);
}
function showSuccess(title, text) {
  $('#successTitle').textContent = title; $('#successText').textContent = text;
  const s = $('#success'); s.classList.remove('show'); void s.offsetWidth; s.clas
  setTimeout(() => s.classList.remove('show'), 2300);
}

let pilgrimFirstRender = true, lastShownRid = null;
const prevStatus = {};
function renderPilgrim() {
  const logged = Session.role === 'pilgrim';
  $('#pLogin').classList.toggle('hidden', logged);
  $('#pHome').classList.toggle('hidden', !logged);
  if (!logged) return;
  const geoTxt = Geo.last ? `<span class="badge b-ok">${T('مفعّل · ±{n} م', { n: G
  const card = $('#pCard');
  card.innerHTML = `
    <div class="who"><div class="avatar"><svg viewBox="0 0 24 24" fill="none" str
      <div><b>${T('حاج في {g}', { g: esc(T(CONFIG.groupName)) })}</b><div style="
```

```
    <div class="kv"><span>${T('المجموعة')}</span>${bdi(CONFIG.group)}</div>
    <div class="kv"><span>${T('المشرفة')}</span><b>${esc(CONFIG.supervisor.name)}
    <div class="kv"><span>${T('الاتصال بالمشرفة')}</span>${connBadge()}</div>
    <div class="kv"><span>${T('الموقع')}</span>${geoTxt}</div>`;
  if (pilgrimFirstRender) { card.classList.remove('flip'); void card.offsetWidth;
  const hasWa = !!waNumber();                                    // WhatsApp only
  $('#pWaBtn').classList.toggle('hidden', !hasWa);
  $('#pContactRow').style.gridTemplateColumns = hasWa ? '' : '1fr';

  $('#pStageBanner').innerHTML = S.stage ? `<div class="banner"><svg viewBox="0 0
  const n = S.notices.filter((x) => Date.now() - x.t < 90 * 60000).sort((a, b) =>
  const nm = n ? MT.show(n.text) : null;      // the supervisor's own words arriv
  $('#pNotice').innerHTML = n ? `<div class="banner notice"><svg viewBox="0 0 24

  const r = myOpenReport() || (() => { const l = myLatestReport(); return l && Da
  $('#pIdle').classList.toggle('hidden', !!r);
  if (!r) { $('#pActive').innerHTML = ''; lastShownRid = null; return; }
  const st = reportStatus(r.rid), s = S.status[r.rid] || {};
  if (prevStatus[r.rid] && prevStatus[r.rid] !== st) {
    if (st === 'seen') { toast(T('✓✓ وصل بلاغك للمشرفة')); haptic(60); }
    if (st === 'ack') { toast(T('🚶 المشرفة في الطريق إليك — خلك مكانك'), 4000);
    if (st === 'resolve') { toast(T('✅ تم إغلاق البلاغ — الحمد لله على السلامة'), 400
  }
  prevStatus[r.rid] = st;
  if (st === 'resolve') stopLiveShare();
  const step = (done, cur, n2, title, sub) => `<li class="${done ? 'done' : cur ?
  const seen = !!s.seen || st === 'ack' || st === 'resolve', ack = !!s.ack || st
  const d = meters(myPos() || r, supPos());
  const card2 = `
    <div class="card ${lastShownRid !== r.rid ? 'flip' : ''}">
      <div style="display:flex;justify-content:space-between;align-items:center;g
        <b style="font-size:16px">${T('بلاغك: {type}', { type: esc(T(r.type)) })}<
        ${done ? `<span class="badge b-ok">${T('مغلق')}</span>` : `<span class="l
      </div>
      <ul class="tl">
        ${step(true, false, 1, T('تم إرسال البلاغ'), T('الساعة {time}', { time: cl
        ${step(seen, !seen, 2, T('وصل للمشرفة'), seen ? T('الساعة {time}', { time
        ${step(ack, seen && !ack, 3, T('المشرفة في الطريق إليك'), ack ? (d != nul
        ${step(done, ack && !done, 4, T('تم إغلاق البلاغ'), done ? T('لله على السلامة
      </ul>
      ${ack && !done && d != null ? `<div style="text-align:center;margin:-4px 0
      ${done ? '' : `${waNumber() ? `<button class="btn wa sm" style="margin-bott
    </div>`;
  $('#pActive').innerHTML = card2;
  lastShownRid = r.rid;
}
```

```
/* ==================== supervisor ==================== */
function supervisorLogin(raw) {
  const ref = normRef(raw);
  if (!isSupRef(ref)) {
    shake('#sRef');
    toast(T(CONFIG.pilgrims.includes(ref) ? 'الرقم المرجعي للمشرفة غير صحيح' : '«
    haptic([100, 50, 100]);
    return;
  }
  if (Session.role === 'pilgrim') leavePilgrim();
  Session.role = 'super'; Session.ref = ref; Session.save();
  Scanner.stop(); Sound.unlock();
  $('#sRef').value = '';
  updatePill();
  $('#sPhone').value = store.get('supPhone', '') || '';
  Relay.connect();
  Relay.publish({ k: 'clear', before: Date.now() - 90000, keepOpenAfter: Date.now
  mapMode = null; gTarget = null; mapFitted = false;
  Geo.watch(onSupFix);
  scheduleRender();
  initMap(); keepAwake(true);
  Push.refresh();
  toast(T('أهلاً {name} 🤍 تم مسح العمليات السابقة — لوحة جديدة', { name: CONFIG.su
  Geo.get().then((p) => Relay.publish(Object.assign({ k: 'hello', role: 'super',
    .catch(() => Relay.publish({ k: 'hello', role: 'super', phone: store.get('sup
}
function leavePilgrim() { stopLiveShare(); }
function leaveSupervisor() { Geo.unwatch(onSupFix); if (supShare) { supShare.canc
function logout() {
  if (!confirm(T('تسجيل الخروج من هذا الجوال؟'))) return;
  if (Session.role === 'pilgrim') leavePilgrim(); else leaveSupervisor();
  Session.clear();
  Relay.disconnect();
  updatePill();
  scheduleRender();
  go('home');
}
function ackReport(rid) {
  Relay.publish(Object.assign({ k: 'ack', rid }, Geo.last || {}));
  hideAlarm();
  toast(T('تم الاستلام — الحاج يشوف الآن إنك في الطريق إليه'));
  syncSupShare();
}
function resolveReport(rid) {
  if (!confirm(T('تأكيد: تم العثور على الحاج وإغلاق البلاغ؟'))) return;
```

```
  Relay.publish({ k: 'resolve', rid });
  syncSupShare();
}
let lastPing = 0;
function pingGroup() {
  if (Date.now() - lastPing < 20000) { toast(T('طلب المواقع قبل لحظات… انتظر شوي
  lastPing = Date.now();
  Relay.publish({ k: 'ping' });
  toast(T('📡 تم طلب مواقع أفراد المجموعة — تظهر خلال ثوانٍ'));
}
function sendNotice(text) {
  text = String(text || '').trim();
  if (!text) { shake('#sNoticeText'); return; }
  Relay.publish({ k: 'notice', text: text.slice(0, 120) });
  $('#sNoticeText').value = '';
  toast(T('📣 وصل التنبيه لأفراد المجموعة'));
}
$$('#sStage button').forEach((b) => b.addEventListener('click', () => {
  Relay.publish({ k: 'stage', stage: b.dataset.s });
  toast(T('🧭 تم تحديد وجهة المجموعة: {s}', { s: T(b.dataset.s) }));
}));
function savePhone() {
  const v = $('#sPhone').value.replace(/[^\d+]/g, '');
  store.set('supPhone', v);
  Relay.publish(Object.assign({ k: 'hello', role: 'super', phone: v }, Geo.last |
  toast(T(v ? 'تم حفظ رقمك — زر الاتصال صار يشتغل عند الحجاج' : 'تم حذف الرقم'));
}
function clearReports() {
  if (!confirm(T('مسح كل البلاغات من اللوحة عند الجميع؟'))) return;
  Relay.publish({ k: 'clear', before: Date.now() });
}
function directionsUrl(p) { return `https://www.google.com/maps/dir/?api=1&destin

let wakeLock = null;
async function keepAwake(on) {
  try {
    if (on && Session.role === 'super' && 'wakeLock' in navigator && !wakeLock &&
      wakeLock = await navigator.wakeLock.request('screen');
      wakeLock.addEventListener('release', () => { wakeLock = null; });
    } else if (!on && wakeLock) { await wakeLock.release(); wakeLock = null; }
  } catch (e) { /* not supported */ }
}

/* new-report alarm on the supervisor phone */
let alarmRid = null, alarmTimer = null;
function checkUnseen() {
```

```
  if (Session.role !== 'super') return;
  const unseen = openReports().filter((r) => !(S.status[r.rid] || {}).seen);
  if (!unseen.length) return;
  unseen.forEach((r) => Relay.publish({ k: 'seen', rid: r.rid }));
  showAlarm(unseen[0]);
}
function showAlarm(r) {
  alarmRid = r.rid;
  fillAlarm(r);
  $('#alarm').classList.add('show');
  Sound.alarm(); haptic([300, 120, 300, 120, 500]);
  let n = 0;
  clearInterval(alarmTimer);
  alarmTimer = setInterval(() => { if (++n > 3 || !$('#alarm').classList.contains
}
function fillAlarm(r) {
  $('#alTitle').textContent = T(r.found ? 'بلاغ جديد' : 'تم العثور على حاج');
  $('#alRef').textContent = r.ref;
  $('#alType').textContent = T(r.type) + ' · ' + T('الساعة {time}', { time: clock
  updateAlarmDist();
}
function updateAlarmDist() {
  const r = S.reports[alarmRid]; if (!r) return;
  const d = meters(supPos(), r);
  $('#alDist').textContent = d != null ? T('يبعد عنك تقريبًا {d}', { d: fmtDist(d)
}
function hideAlarm() { $('#alarm').classList.remove('show'); clearInterval(alarmT
function alarmAck() { const rid = alarmRid; hideAlarm(); if (current !== 'super')
function alarmShow() { const rid = alarmRid; hideAlarm(); if (current !== 'super'

function connBadge() { return Relay.online ? `<span class="conn on"><i></i><span>
function renderConn() {
  const c = $('#sConn');
  if (c) { c.classList.toggle('on', Relay.online); c.querySelector('span').textCo
  scheduleRender();
}

let lastReportsHTML = '';
function renderSupervisor() {
  const logged = Session.role === 'super';
  $('#sLogin').classList.toggle('hidden', logged);
  $('#sDash').classList.toggle('hidden', !logged);
  if (!logged) return;
  $('#sName').innerHTML = esc(CONFIG.supervisor.name) + ' · ' + bdi(CONFIG.superv
  $('#sGroup').innerHTML = T('المجموعة {g} · {n} حجاج', { g: bdi(CONFIG.group), n
  const me = supPos();
```

```
  const online = CONFIG.pilgrims.filter((r) => isOnline(S.members[r])).length;
  const open = openReports();
  let far = null;
  CONFIG.pilgrims.forEach((ref) => { const d = meters(me, openReportOf(ref) || th
  setStat('#stOnline', online + '/' + CONFIG.pilgrims.length);
  setStat('#stOpen', String(open.length));
  setStat('#stFar', far == null ? '—' : fmtDist(far));

  const all = Object.values(S.reports).filter(visibleReport).sort((a, b) => (isOp
  const html = all.length ? all.map((r) => {
    const st = reportStatus(r.rid), d = meters(me, r);
    const badge = { sent: ['جديد', 'b-alert'], seen: ['جديد', 'b-alert'], ack: ['
    const acts = st === 'sent' || st === 'seen'
      ? `<button class="btn red" onclick="ackReport('${r.rid}')">${T('تلام والتوجه
      : st === 'ack' ? `<button class="btn green" onclick="resolveReport('${r.rid
    const loc = r.lat != null;
    return `<div class="rep ${isOpen(r) ? '' : 'closed'} ${Date.now() - r.t < 800
      <div class="l1">${bdi(r.ref)}<span class="badge ${badge[1]}">${T(badge[0])}
      <div class="l2"><span>${esc(T(r.type))} · ${T('الساعة {time}', { time: cloc
      ${isOpen(r) ? `<div class="acts">${acts}${loc ? `<button class="btn ghost"
    </div>`;
  }).join('') : `<div class="empty">${T('🤍 لا توجد بلاغات — كل الحجاج بخير')}</div
  if (html !== lastReportsHTML) { $('#sReports').innerHTML = html; lastReportsHTM

  $('#sMembers').innerHTML = CONFIG.pilgrims.map((ref) => {
    const m = thisSession(S.members[ref]), rep = openReportOf(ref), d = meters(me
    const badge = rep ? `<span class="badge b-alert">${T('عليه بلاغ')}</span>` : i
    return `<div class="li"><div>${bdi(ref)}<div class="meta">${d != null ? T('نك
  }).join('');
  $$('#sStage button').forEach((b) => b.classList.toggle('on', !!S.stage && S.sta
  if (!mapMode) setMapMode(openReports().some((r) => r.lat != null) ? 'google' :
  renderMap();
  renderGoogle(false);
  renderNotif();
  if (alarmRid && $('#alarm').classList.contains('show')) updateAlarmDist();
}
function setStat(sel, val) {
  const el = $(sel); if (el.textContent === val) return;
  el.textContent = val; el.animate && el.animate([{ transform: 'scale(1.25)' }, {
}

/* ==================== maps: Google Maps view of a report + live group dots (Le
let map = null, mapFitted = false, mapMode = null, gTarget = null;
const marks = {}, lines = {};
async function initMap() {
  if (map) { setTimeout(() => map.invalidateSize(), 80); return; }
```

```
  loadCSS('leaflet.css');
  try { await loadScript('leaflet.js'); } catch (e) { $('#mapMsg').textContent =
  if (map) return;
  map = window.L.map('lmap', { zoomControl: false, attributionControl: true }).se
  window.L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
    maxZoom: 19, attribution: '© OpenStreetMap',
  }).addTo(map);
  window.L.control.zoom({ position: 'bottomleft' }).addTo(map);
  map.attributionControl.setPrefix(false);
  setTimeout(() => map.invalidateSize(), 120);
  renderMap();
}
const SHIELD = '<svg viewBox="0 0 24 24" fill="none" stroke="#fff" stroke-width="
function upsertMark(key, pos, cls, html, tip, z = 0) {
  const L = window.L;
  const icon = L.divIcon({ className: 'mkwrap', html: `<div class="${cls}">${html
  let m = marks[key];
  if (!m) { m = marks[key] = L.marker([pos.lat, pos.lon], { icon, zIndexOffset: z
  else {
    m.setLatLng([pos.lat, pos.lon]);
    if (m._sig !== cls + html) { m.setIcon(icon); m._sig = cls + html; m.setZInde
    m.setTooltipContent(tip);
  }
}
function dropMark(key) { if (marks[key]) { marks[key].remove(); delete marks[key]
function renderMap() {
  if (!map || Session.role !== 'super') return;
  const L = window.L, me = supPos();
  const seen = new Set();
  if (me) { upsertMark('sup', me, 'mk mk-sup', SHIELD, T('(المشرفة) أنت'), 500);
  CONFIG.pilgrims.forEach((ref) => {
    const m = thisSession(S.members[ref]), rep = openReportOf(ref), pos = rep &&
    if (!pos) return;
    const d = meters(me, pos);
    const tip = (rep ? '🧯 ' + T(rep.type) + ' — ' : '') + ref + (d != null ? ' ·
    upsertMark(ref, pos, rep ? 'mk mk-rep' : 'mk mk-pil' + (isOnline(m) ? '' : '
    seen.add(ref);
  });
  Object.keys(marks).forEach((k) => { if (!seen.has(k)) dropMark(k); });
  // a dashed line from the supervisor to every open report + the pilgrim's live
  const want = new Set();
  openReports().forEach((r) => {
    if (r.lat == null) return;
    if (me) {
      want.add('l' + r.rid);
      const ll = [[me.lat, me.lon], [r.lat, r.lon]];
```

```
      if (lines['l' + r.rid]) lines['l' + r.rid].setLatLngs(ll);
      else lines['l' + r.rid] = L.polyline(ll, { color: '#A6402F', weight: 3, das
    }
    if (r.trail.length > 1) {
      want.add('t' + r.rid);
      if (lines['t' + r.rid]) lines['t' + r.rid].setLatLngs(r.trail);
      else lines['t' + r.rid] = L.polyline(r.trail, { color: '#C0553F', weight: 4
    }
  });
  Object.keys(lines).forEach((k) => { if (!want.has(k)) { lines[k].remove(); dele
  const n = Object.keys(marks).length;
  const msg = $('#mapMsg');
  msg.classList.toggle('hidden', n > 0);
  if (!n) msg.textContent = T('المواقع… اسمح بالموقع واضغط «تحديث مواقع المجموعة
  if (!mapFitted && n) { fitMap(); mapFitted = true; }
}
function fitMap() {
  if (!map) return;
  const pts = Object.values(marks).map((m) => m.getLatLng());
  if (!pts.length) { toast(T('ما فيه مواقع بعد')); return; }
  if (pts.length === 1) map.setView(pts[0], 18, { animate: true });
  else map.fitBounds(window.L.latLngBounds(pts), { padding: [56, 56], maxZoom: 19
}
function focusReport(rid) {
  const r = S.reports[rid]; if (!r) return;
  setMapMode('google', rid);
  $('#mapSeg').scrollIntoView({ behavior: 'smooth', block: 'start' });
  if (map && r.lat != null) map.setView([r.lat, r.lon], 18);
}
function setMapMode(mode, rid) {
  mapMode = mode;
  if (rid) gTarget = rid;
  $$('#mapSeg button').forEach((b) => b.classList.toggle('on', b.dataset.m === mo
  $('.mapwrap').classList.toggle('google', mode === 'google');
  $('#gmap').classList.toggle('hidden', mode !== 'google');
  if (mode === 'group' && map) setTimeout(() => { map.invalidateSize(); fitMap();
  renderGoogle(true);
}
function googleTarget() {              // the chosen report, else the newest open
  let r = gTarget && S.reports[gTarget];
  if (!r || r.lat == null || !isOpen(r)) r = openReports().find((x) => x.lat != n
  if (r) return { lat: r.lat, lon: r.lon, label: T(r.found ? 'ان العثور على الحاج
  const me = supPos();
  return me ? { lat: me.lat, lon: me.lon, label: T('(المشرفة) موقعك الحالي') } :
}
function renderGoogle(force) {
```

```
  const cap = $('#gcap'), box = $('#gmap');
  if (mapMode !== 'google') { cap.classList.remove('show'); return; }
  const t = googleTarget();
  if (!t) { box.dataset.key = ''; box.innerHTML = `<div class="mapmsg">${T('جوالك
  const key = I18N.lang + ':' + t.lat.toFixed(5) + ',' + t.lon.toFixed(5);
  if (force || box.dataset.key !== key) {
    box.dataset.key = key;
    box.innerHTML = `<iframe title="${esc(T('خرائط جوجل'))}" loading="lazy" src="
  }
  const d = t.r ? meters(supPos(), t.r) : null;
  cap.innerHTML = `<div class="t"><b>${esc(t.label)}</b>${d != null ? `<span clas
    <div class="links"><a class="glink" href="${gmapsSearch(t)}" target="_blank"
  cap.classList.add('show');
}

/* ==================== home: stage + route ==================== */
function renderHome() {
  const st = S.stage ? S.stage.stage : null;
  const idx = st ? CONFIG.stages.indexOf(st) : 1;
  $('#nowText').textContent = st ? T('وجهة المجموعة الآن: {s}', { s: T(st) }) : T(
  $$('#routeSteps .step').forEach((s) => {
    const i = +s.dataset.i;
    s.classList.toggle('done', i < idx); s.classList.toggle('cur', i === idx);
  });
  $('#routeNote').textContent = st ? T('حددتها المشرفة {ago}', { ago: ago(S.stage
  routeIdx = idx;
}
let routeIdx = 1;
function runRoute() {
  const f = $('#fill'); f.style.width = '0';
  setTimeout(() => { f.style.width = `calc((100% - 44px) * ${routeIdx / 3})`; },
}

/* ==================== render loop ==================== */
let renderQueued = false;
function scheduleRender() {
  if (renderQueued) return;
  renderQueued = true;
  requestAnimationFrame(() => {
    renderQueued = false;
    const prevIdx = routeIdx;
    renderHome();
    if (prevIdx !== routeIdx && current === 'home') runRoute();
    renderPilgrim();
    renderSupervisor();
    if (current === 'places') { renderPlaces(); if (Geo.last && mlState !== 'ok'
```

```
    checkUnseen();
    syncSupShare();
  });
}
setInterval(() => { if (Session.role) scheduleRender(); }, 30000);   // refresh "

/* ==================== 3D Kaaba ==================== */
let k3d = null, k3dFullOpen = false, heroVisible = true;
function initK3D() {
  import('./kaaba3d.js?v=5').then((mod) => {
    k3d = mod.createKaaba($('#k3d'), { onInteract: () => $('#k3dHint').classList.
    if (!k3d) return;                          // no WebGL: the CSS Kaaba stays
    $('#k3d').classList.add('ready');
    updateK3dActive();
  }).catch(() => { /* CSS Kaaba stays */ });
}
function updateK3dActive() { if (k3d) k3d.setActive(!document.hidden && (k3dFullO
function openK3dFull() {
  if (!k3d) { toast(T('العرض ثلاثي الأبعاد غير مدعوم في هذا المتصفح')); return; }
  k3dFullOpen = true;
  $('#k3dFull').classList.add('show');
  k3d.moveTo($('#k3dStage'), 'full');
  k3d.setLabels($('#k3dLabels'));
  I18N.apply($('#k3dLabels'));
  updateK3dActive();
  history.pushState({ k3d: 1 }, '');
}
function closeK3dFull(fromPop) {
  if (!k3dFullOpen) return;
  k3dFullOpen = false;
  $('#k3dFull').classList.remove('show');
  k3d.setLabels(null);
  k3d.moveTo($('#k3d'), 'inline');
  k3d.resetView();
  updateK3dActive();
  if (!fromPop && history.state && history.state.k3d) history.back();
}
function toggleTawaf() {
  if (!k3d) return;
  k3d.setTawaf(!k3d.tawaf);
  $('#tawafBtn span').textContent = T(k3d.tawaf ? 'تشغيل الطواف' : 'إيقاف الطواف'
}

/* ==================== install + share ==================== */
let deferredPrompt = null;
const isStandalone = () => matchMedia('(display-mode: standalone)').matches || na
```

```
const isIOS = () => /iphone|ipad|ipod/i.test(navigator.userAgent) || (navigator.p
window.addEventListener('beforeinstallprompt', (e) => { e.preventDefault(); defer
window.addEventListener('appinstalled', () => $('#installCard').classList.add('hi
async function installApp() {
  if (deferredPrompt) {
    deferredPrompt.prompt();
    try { const r = await deferredPrompt.userChoice; if (r.outcome === 'accepted'
    deferredPrompt = null;
  } else if (isIOS()) show('iosSheet');
  else toast(T('قائمة المتصفح اختر «تثبيت التطبيق» أو «إضافة إلى الشاشة الرئيسية
}
const appUrl = () => location.href.split('#')[0].split('?')[0].replace(/index\.ht
async function openShare() {
  try { await loadScript('qrcode.js'); } catch (e) { toast(T('تعذّر إنشاء الرمز'))
  const url = appUrl();
  const qr = window.qrcode(0, 'M'); qr.addData(url); qr.make();
  $('#shareQR').innerHTML = qr.createSvgTag({ cellSize: 6, margin: 2, scalable: t
  $('#shareURL').textContent = url;
  show('shareSheet');
}
async function copyLink() {
  try { await navigator.clipboard.writeText(appUrl()); toast(T('✓ تم نسخ الرابط'))
}
async function nativeShare() {
  if (navigator.share) { try { await navigator.share({ title: T('درب الحاج'), tex
  else copyLink();
}

/* ==================== UI sugar ==================== */
document.addEventListener('click', (e) => {               // ripple on buttons
  const b = e.target.closest('.btn'); if (!b) return;
  const r = document.createElement('span'), d = Math.max(b.clientWidth, b.clientH
  r.className = 'ripple'; r.style.width = r.style.height = d + 'px';
  r.style.left = (e.clientX - rect.left - d / 2) + 'px'; r.style.top = (e.clientY
  b.appendChild(r); setTimeout(() => r.remove(), 600);
});
$$('.tilt').forEach((el) => {                             // 3D tilt on touch
  const move = (x, y) => {
    const r = el.getBoundingClientRect();
    const rx = ((y - r.top) / r.height - 0.5) * -14, ry = ((x - r.left) / r.width
    el.style.transform = `perspective(600px) rotateX(${rx}deg) rotateY(${ry}deg)
  };
  el.addEventListener('pointermove', (e) => move(e.clientX, e.clientY));
  el.addEventListener('touchmove', (e) => move(e.touches[0].clientX, e.touches[0]
  ['pointerleave', 'touchend'].forEach((ev) => el.addEventListener(ev, () => { el
});
```

```
$('#app').addEventListener('scroll', (e) => { if (e.target.scrollTop || e.target.
$('#pRef').addEventListener('keydown', (e) => { if (e.key === 'Enter') pilgrimLog
$('#sRef').addEventListener('keydown', (e) => { if (e.key === 'Enter') supervisor
$('#sNoticeText').addEventListener('keydown', (e) => { if (e.key === 'Enter') sen

let hiddenAt = 0;
document.addEventListener('visibilitychange', () => {
  updateK3dActive();
  if (document.hidden) { hiddenAt = Date.now(); Scanner.stop(); return; }
  if (Session.role && Date.now() - hiddenAt > 20000) Relay.refresh();
  if (Session.role === 'super' && current === 'super') keepAwake(true);
  if (Session.role === 'pilgrim' && Date.now() - hiddenAt > 5 * 60000 && Date.now
    Geo.get().then((p) => Relay.publish(Object.assign({ k: 'hello', role: 'pilgri
  }
  Relay.flush();
});
window.addEventListener('online', () => { Relay.flush(); Relay.refresh(); });



/* ==================== icons used in rendered markup ==================== */
const ICON = {
  pin: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
  nav: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
  wa: '<svg viewBox="0 0 24 24" fill="currentColor"><path d="M12 2a10 10 0 0 0-8.
  phone: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
  mail: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width=
  chev: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width=
};

/* ==================== WhatsApp + phone helpers ==================== */
function intlNumber(n) {
  n = String(n || '').replace(/[٩-٠]/g, (d) => String('٠١٢٣٤٥٦٧٨٩'.indexOf(d))).r
  if (!n) return '';
  if (n.startsWith('+')) n = n.slice(1);
  if (n.startsWith('00')) n = n.slice(2);
  if (n.startsWith('05')) n = '966' + n.slice(1);
  else if (/^5\d{8}$/.test(n)) n = '966' + n;
  return n;
}
const waNumber = () => intlNumber(CONFIG.contact.phone || S.sup.phone);
const waLink = (text, number = waNumber()) => `https://wa.me/${number}?text=${enc
const gmapsSearch = (p) => `https://www.google.com/maps/search/?api=1&query=${p.l
const gmapsDir = (p) => `https://www.google.com/maps/dir/?api=1&destination=${p.l
function openExternal(url) {
  const w = window.open(url, '_blank', 'noopener');
```

```
  if (!w) location.href = url;
}
function withFreshLocation(cb) {                  // use a recent fix right away,
  if (Geo.last && Date.now() - (Geo.lastAt || 0) < 120000) { cb(Geo.last); return
  toast(T('جارٍ تحديد موقعك…'));
  Geo.get(10000).then(cb).catch(() => cb(null));
}
function whatsappHelp() {
  if (!waNumber()) { toast(T('لواتساب غير مضاف بعد — ارفع بلاغ وبيوصل للمشرفة فورًا
  const r = myOpenReport();
  withFreshLocation((p) => {
    const text = `السلام عليكم، أنا الحاج صاحب الرقم المرجعي ${Session.ref} من ${C
      + (r ? ` رفعت بلاغ: ${r.type}.` : ' أحتاج مساعدة.')
      + (p ? ` موقعي الحالي: ${gmapsSearch(p)}` : '');
    openExternal(waLink(text));
  });
}

/* ==================== location page: my current location + the group's schedul
async function geoPermission() {
  try { if (navigator.permissions && navigator.permissions.query) return (await n
  return 'unknown';
}
function currentStop() {
  const st = S.stage ? S.stage.stage : 'منى';
  const i = CONFIG.schedule.findIndex((x) => x.stage === st);
  return i < 0 ? 1 : i;
}
function renderPlaces() {
  const now = currentStop(), me = Geo.last, cur = CONFIG.schedule[now];
  $('#plNowTitle').textContent = T('الآن: {x}', { x: (cur.tag ? T(cur.tag) + ' · '
  $('#plNowSub').textContent = S.stage ? T('حددتها المشرفة {ago}', { ago: ago(S.s
  $('#plList').innerHTML = CONFIG.schedule.map((x, i) => {
    const cls = i < now ? 'st-done' : i === now ? 'st-now' : '';
    const d = meters(me, x);
    const badges = (i === now ? `<span class="badge b-ok">${T('المجموعة هنا الآن')
      + (d != null ? `<span class="badge b-away">${T('يبعد عنك {d}', { d: fmtDist
    return `<div class="stop ${cls}"><div class="rail"><i class="d"></i><i class=
      <div class="when"><b>${T(x.day)}</b><span>· ${T(x.time)}</span>${x.tag ? `<
      <div class="name">${T(x.name)}</div><div class="note">${T(x.note)}</div>
      ${badges ? `<div class="meta">${badges}</div>` : ''}
      <div class="links"><a class="glink" href="${gmapsSearch(x)}" target="_blank
    </div></div>`;
  }).join('');
}
let mlState = 'idle';
```

```
async function onPlacesOpen() {
  renderPlaces();
  if (Geo.last) { renderMyLoc('ok'); return; }
  if (!window.isSecureContext) { renderMyLoc('insecure'); return; }
  const perm = await geoPermission();
  if (perm === 'granted') locateMe(true);
  else if (perm === 'denied') renderMyLoc('denied');
  else renderMyLoc('idle');
}
function locateMe(silent) {
  if (!window.isSecureContext) { renderMyLoc('insecure'); return; }
  if (!navigator.geolocation) { renderMyLoc('unsupported'); return; }
  renderMyLoc('loading');
  Geo.get(0).then(() => { renderMyLoc('ok'); renderPlaces(); if (!silent) haptic(
    .catch((e) => renderMyLoc(e && e.code === 1 ? 'denied' : e && e.code === 3 ?
}
function renderMyLoc(state) {
  mlState = state;
  const st = $('#mlStatus'), body = $('#mlBody'), btn = $('#mlBtn');
  btn.disabled = state === 'loading';
  btn.textContent = T(state === 'ok' ? 'تحديث' : state === 'loading' ? 'دﻳﺪ ﻣﻮﻗﻌﻲ'
  if (state === 'ok') {
    const p = Geo.last;
    st.textContent = p.acc > 500 ? T('ﻣﻮﻗﻌﻚ ﺗﻘﺮﻳﺒﻲ (±{d}) — أﻋﻠﻰ ﻟﺪﻗﺔ «ﻮﻗﻊ اﻟﺪﻗﻴﻖ
    const key = I18N.lang + ':' + p.lat + ',' + p.lon;
    if (body.dataset.key !== key) {
      body.dataset.key = key;
      body.innerHTML = `<div class="mlmap"><iframe loading="lazy" title="${esc(T(
        <div class="coords"><span>${T('الإحداثيات')}</span><bdi>${p.lat.toFixed(5)
        <div style="display:grid;grid-template-columns:1fr 1fr;gap:8px;margin-top
    }
    return;
  }
  body.dataset.key = '';
  st.textContent = T({
    idle: 'اضغط «تحديد موقعي» واسمح بالوصول للموقع',
    loading: 'جارٍ تحديد موقعك… (قد يأخذ ثواني)',
    denied: 'الوصول للموقع مقفول — فعّله بالخطوات التالية',
    timeout: 'ما قدر الجوال يحدد موقعك',
    fail: 'تعذّر تحديد الموقع',
    insecure: 'الموقع يشتغل فقط من رابط التطبيق (https)',
    unsupported: 'المتصفح لا يدعم تحديد الموقع',
  }[state]);
  body.innerHTML = state === 'loading' || state === 'idle' ? '' : geoHelp(state);
}
// help blocks are translated as whole pieces (they contain <b> tags)
```

```
const GEO_HELP = {
  insecure: 'افتح التطبيق من رابطه على الإنترنت (يبدأ بـ https مثل رابط Netlify).',
  ios: '<li>افتح <b>الإعدادات</b> ← <b>الخصوصية والأمان</b> ← <b>خدمات الموقع</b> ة
  other: '<li>أو اضغط ⓘ بجانب الرابط فوق 🔒.</li><li>اختر <b>الأذونات</b> ← <b>قع
  fail: 'تأكد إن الموقع (GPS) مرة ثانية «تحديد موقعي» ثم اضغط فذة أو خارج المبنى،
  browser: 'استخدم Safari أو Chrome في الآيفون والآيباد، في الأندرويد.',
};
function geoHelp(state) {
  if (state === 'insecure') return `<div class="help">${T(GEO_HELP.insecure)}</di
  if (state === 'denied') return `<ol class="help">${T(isIOS() ? GEO_HELP.ios : G
  if (state === 'timeout' || state === 'fail') return `<div class="help">${T(GEO_
  return `<div class="help">${T(GEO_HELP.browser)}</div>`;
}
async function shareLocationLink() {
  const p = Geo.last; if (!p) return;
  const url = gmapsSearch(p);
  if (navigator.share) { try { await navigator.share({ title: T('وقعي – درب الحاج
  openExternal('https://wa.me/?text=' + encodeURIComponent(T('موقعي الحالي: {url}
}
function enableLocation() {
  if (!window.isSecureContext) { go('places'); setTimeout(() => renderMyLoc('inse
  Geo.get(0).then((p) => {
    toast(T('📍 تم تفعيل الموقع'));
    if (Session.role === 'pilgrim') Relay.publish(Object.assign({ k: 'loc' }, p))
  }).catch((e) => { go('places'); setTimeout(() => renderMyLoc(e && e.code === 1
}

/* ==================== scan tab: anyone who finds a lost pilgrim ==============
function scanFinder() { Scanner.start($('#fScanner'), (code) => finderLookup(code
let lastFinder = null;                          // last result card, redrawn when
function finderLookup(raw) {
  const ref = normRef(raw);
  lastFinder = ref;
  if (CONFIG.pilgrims.includes(ref)) $('#fRef').value = '';
  renderFinder(ref, true);
}
function renderFinder(ref, fx) {
  const box = $('#fResult'), flip = fx ? ' flip' : '';
  if (isSupRef(ref)) {
    box.innerHTML = `<div class="card${flip} fres"><div class="head"><b>${T('شرفة
      <div class="kv"><span>${T('المشرفة')}</span><b>${esc(CONFIG.supervisor.name
      <div class="kv"><span>${T('المجموعة')}</span>${bdi(CONFIG.group)}</div>
      <div class="acts"><button class="btn green full" onclick="loginFromScan('su
    if (fx) haptic(40);
    return;
  }
```

```
  if (!CONFIG.pilgrims.includes(ref)) {
    if (fx) { shake('#fRef'); haptic([100, 50, 100]); }
    box.innerHTML = `<div class="card${fx ? ' shake' : ''}" style="text-align:cen
    return;
  }
  const rep = openReportOf(ref);
  box.innerHTML = `<div class="card${flip} fres">
    <div class="head"><b>${T('تم التعرّف على الحاج')}</b><span class="badge b-ok">
    <div class="kv"><span>${T('الرقم المرجعي')}</span>${bdi(ref)}</div>
    <div class="kv"><span>${T('المجموعة')}</span>${bdi(CONFIG.group)}</div>
    <div class="kv"><span>${T('المشرفة المسؤولة')}</span><b>${esc(CONFIG.supervis
    ${rep ? `<div class="kv"><span>${T('الحالة')}</span><span class="badge b-aler
    <div class="acts">
      <button class="btn red full" onclick="reportFound('${ref}')">${T('هكانه الآن
      ${waNumber() ? `<button class="btn wa" onclick="whatsappFound('${ref}')">${
      ${Session.role === 'pilgrim' && Session.ref === ref ? '' : `<button class="
    </div></div>`;
  if (fx) haptic(40);
}
function loginFromScan(role, ref) {
  go(role);
  setTimeout(() => (role === 'pilgrim' ? pilgrimLogin(ref) : supervisorLogin(ref)
}
function reportFound(ref) {
  withFreshLocation((p) => {
    Relay.publish(Object.assign({ k: 'found', rid: uid(), ref }, p || {}));
    publishAlert('found', ref, null, p);
    showSuccess(T('تم إبلاغ المشرفة'), T(p ? 'لبلاغ — الموقع غير متاح، خلك مع الحاج
    haptic([60, 40, 60]);
  });
}
function whatsappFound(ref) {
  withFreshLocation((p) => {
    const text = `السلام عليكم، لقيت الحاج صاحب الرقم المرجعي ${ref} من ${CONFIG.g
    openExternal(waLink(text));
  });
}
$('#fRef').addEventListener('keydown', (e) => { if (e.key === 'Enter') finderLook

/* ==================== team & contact ==================== */
function renderContact() {
  $('#teamList').innerHTML = CONFIG.team.map((m) => `<div class="member"><div cla
    <div class="m-name">${esc(m.name)}<span class="m-role">${esc(T(m.role))}</spa
  const row = (cls, icon, label, value, href) => `<a class="crow" href="${href}"$
    <div class="c-ic ${cls}">${icon}</div><div><div class="c-lab">${label}</div><
  const ph = CONFIG.contact.phone, rows = [];
```

```
  if (ph) {
    rows.push(row('wa', ICON.wa, T('واتساب'), ph, waLink('م بخصوص تطبيق درب الحاج
    rows.push(row('ph', ICON.phone, T('اتصال'), ph, 'tel:' + ph));
  }
  rows.push(row('em', ICON.mail, T('البريد الإلكتروني'), CONFIG.contact.email, `ma
  $('#contactList').innerHTML = rows.join('');
  $('#emgGrid').innerHTML = CONFIG.emergency.map((e) => `<a class="emg" href="tel
}


/* ==================== alerts that ring the supervisor's phone even when the ap
// Every report also goes to a quiet "alerts" channel. The supervisor's phone get
//  • Web Push (the installed app, through ntfy.sh's push service), or
//  • the free ntfy app subscribed to the same channel (works on every phone).
function publishAlert(kind, ref, type, p) {
  const test = kind === 'test', found = kind === 'found';
  const title = test ? 'تنبيه تجريبي — درب الحاج' : found ? 'بلاغ جديد — درب الحاج
  const body = test ? 'تنبيهات شغّالة ✓ أي بلاغ جديد بيوصلك هنا حتى لو التطبيق مقفل
    : (found ? `شخص لقى الحاج ${ref}` : `وبلّغ بمكانه ${ref}: ${type}`) + (p
  const q = new URLSearchParams({ title, priority: test ? 'high' : 'urgent', tags
  return fetch(`${CONFIG.relay.replace(/\/$/, '')}/${CONFIG.alertTopic}?${q}`, {
}
function toB64u(buf) { return btoa(String.fromCharCode(...new Uint8Array(buf))).r
function b64u(str) {
  const pad = '='.repeat((4 - (str.length % 4)) % 4);
  const raw = atob((str + pad).replace(/-/g, '+').replace(/_/g, '/'));
  return Uint8Array.from(raw, (c) => c.charCodeAt(0));
}
const Push = {
  supported() { return 'serviceWorker' in navigator && 'PushManager' in window &&
  isOn() { return store.get('pushOn', false) && 'Notification' in window && Notif
  key() {                                    // ntfy.sh's public Web Push key, read
    if (this._key) return Promise.resolve(this._key);
    return new Promise((res, rej) => {
      const f = document.createElement('iframe');
      f.style.display = 'none';
      f.setAttribute('sandbox', 'allow-scripts');
      let to = 0;
      const done = (k) => { window.removeEventListener('message', onMsg); clearTi
      const onMsg = (e) => { if (e.source === f.contentWindow) done(typeof e.data
      window.addEventListener('message', onMsg);
      to = setTimeout(() => done(null), 12000);
      f.srcdoc = `<script src="${CONFIG.relay.replace(/\/$/, '')}/config.js"><\/s
      document.body.appendChild(f);
    });
  },
```

```
  async subscribe() {
    const reg = await navigator.serviceWorker.ready;
    const key = await this.key();
    let sub = await reg.pushManager.getSubscription();
    const cur = sub && sub.options && sub.options.applicationServerKey;
    if (sub && cur && toB64u(cur) !== key.replace(/=+$/, '')) { await sub.unsubsc
    if (!sub) sub = await reg.pushManager.subscribe({ userVisibleOnly: true, appl
    const j = sub.toJSON();
    await fetch(CONFIG.relay.replace(/\/$/, '') + '/v1/webpush', {
      method: 'POST', mode: 'no-cors',
      body: JSON.stringify({ endpoint: j.endpoint, auth: j.keys.auth, p256dh: j.k
    });
    store.set('pushOn', true);
  },
  async enable() {
    if (!this.supported()) throw new Error(isIOS() && !isStandalone() ? 'install'
    const perm = await Notification.requestPermission();      // must be the firs
    if (perm !== 'granted') throw new Error('denied');
    await this.subscribe();
  },
  refresh() { if (this.isOn() && this.supported()) this.subscribe().catch(() => {
};
let pushTestAt = 0, pushTestOk = false;
async function enableAlerts() {
  const btn = $('#nEnable'); if (btn) { btn.disabled = true; btn.textContent = T(
  try {
    await Push.enable();
    toast(T('🔔 «تنبيه تجريبي — جربي» تم تفعيل التنبيهات'), 3500);
  } catch (e) {
    const m = e && e.message;
    if (m === 'install') show('iosSheet');
    toast(T(m === 'install' ? 'الرئيسية وافتحيه من الأيقونة، بعدين فعّلي التنبيهات'
      : m === 'denied' ? 'التنبيهات مرفوضة — فعّليها من الإعدادات ← الإشعارات ← درب الحاج'
      : m === 'unsupported' ? 'هذا المتصفح ما يدعم التنبيهات — استخدمي تطبيق ntfy'
      : 'استخدمي تطبيق ntfy ما قدرت أفعّلها هنا (الطريقة المضمونة)'), 4500);
  }
  notifSig = '';
  renderNotif();
}
function testAlert() {
  pushTestAt = Date.now(); pushTestOk = false;
  publishAlert('test');
  toast(T('أرسلت تنبيه تجريبي 🔔 يوصلك خلال ثواني'), 3500);
  setTimeout(() => { if (!pushTestOk && Date.now() - pushTestAt >= 24000 && Push.
}
async function copyTopic() {
```

```
  try { await navigator.clipboard.writeText(CONFIG.alertTopic); toast(T('تم نسخ القناة
}
let notifSig = '';
function renderNotif() {
  const box = $('#sNotif'); if (!box) return;
  const on = Push.isOn(), needInstall = isIOS() && !isStandalone();
  const sig = [on, needInstall, Push.supported(), I18N.lang].join();
  if (sig === notifSig) return;
  notifSig = sig;
  box.classList.toggle('ok', on);
  const bell = '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
  const head = on
    ? `<div class="nh"><div class="nic">${bell}</div><div class="nt"><b>${T('فعّلة
    : `<div class="nh"><div class="nic">${bell}</div><div class="nt"><b>${T('مقفل
  const acts = on
    ? `<div class="acts"><button class="btn gold" onclick="testAlert()">${T('جريبي
    : needInstall
      ? `<div class="acts"><button class="btn gold full" onclick="show('iosSheet'
      : `<div class="acts"><button class="btn red full" id="nEnable" onclick="ena
  const ntfy = `<details><summary>${T('تطبيق: المضمونة الطريقة ntfy المجاني')}</s
    <ol class="help" style="margin-top:8px"><li>${T('نزّلي تطبيق <b>ntfy</b> من Ap
    <div class="topic"><code>${CONFIG.alertTopic}</code><button class="btn gold"
    <div class="sub" style="margin:8px 0 10px;font-size:12.5px">${T('خلّي إشعارات
    <button class="btn ghost sm" onclick="testAlert()">${T('إرسال تنبيه تجريبي')}
  box.innerHTML = head + acts + ntfy;
}

/* the app was opened from a notification → go straight to the dashboard */
function openFromNotification() {
  if (Session.role === 'super' && current !== 'super') go('super');
}
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.addEventListener('message', (e) => {
    const d = e.data || {};
    if (d.type === 'push') { pushTestOk = true; if (Date.now() - pushTestAt < 600
    if (d.type === 'open') openFromNotification();
  });
}
window.addEventListener('hashchange', () => { if (location.hash === '#super') ope

/* ==================== live translation of what the supervisor types ==========
// Built-in sentences (buttons, stages, preset notices) come from i18n.js. Free t
// is translated on each pilgrim's phone into that phone's language: MyMemory (fr
// If both are unreachable the original text is shown, exactly as before.
function decodeHtml(s) { const d = document.createElement('textarea'); d.innerHTM
async function translateOnline(text, sl, tl) {
```

```
  const get = async (url, pick) => {
    const ctl = window.AbortController ? new AbortController() : null;
    const to = setTimeout(() => { if (ctl) ctl.abort(); }, 7000);
    try {
      const r = await fetch(url, ctl ? { signal: ctl.signal } : {});
      if (!r.ok) throw new Error('http ' + r.status);
      return pick(await r.json());
    } finally { clearTimeout(to); }
  };
  const q = encodeURIComponent(text);
  const tries = [
    () => get(`https://api.mymemory.translated.net/get?q=${q}&langpair=${encodeUR
      const t = j && j.responseData && j.responseData.translatedText;
      if (!t || Number(j.responseStatus) !== 200 || /MYMEMORY WARNING|QUERY LENGT
      return decodeHtml(t);
    }),
    () => get(`https://translate.googleapis.com/translate_a/single?client=gtx&dt=
      const t = Array.isArray(j) && Array.isArray(j[0]) ? j[0].map((x) => (x && x
      if (!t) throw new Error('google');
      return t;
    }),
    () => get(`https://lingva.ml/api/v1/${sl}/${tl}/${q}`, (j) => {
      if (!j || !j.translation) throw new Error('lingva');
      return j.translation;
    }),
  ];
  // a "translation" that is still the same text (or still Arabic for a Latin/Cyr
  const latinOrOther = ['en', 'id', 'tr', 'fr', 'ms', 'ru', 'bn'].includes(tl);
  const good = (t) => !!t && !!t.trim() && t.trim() !== text.trim() && !(latinOrO
  for (const f of tries) { try { const t = await f(); if (good(t)) return t.trim(
  return null;
}
const MT = {
  cache: (() => { const c = store.get('mt', {}); return c && typeof c === 'object
  busy: {},
  src: (text) => (/[ء-ۿ]/.test(text) ? 'ar' : 'en'),
  /** what to show right now; starts a translation in the background when needed
  show(text) {
    text = String(text || '');
    const dict = T(text);
    if (dict !== text || !text.trim()) return { text: dict, auto: false };
    const tl = I18N.lang, sl = this.src(text);
    if (tl === sl) return { text, auto: false };
    const hit = this.cache[tl + '|' + text];
    if (hit) return { text: hit, auto: true };
    this.fetch(text, sl, tl);
```

```
    return { text, auto: false };
  },
  /** the best text within ms (used for the toast) */
  ready(text, ms = 3500) {
    const now = this.show(text);
    const tl = I18N.lang, sl = this.src(text), k = tl + '|' + text;
    if (now.auto || now.text !== text || tl === sl || !text.trim()) return Promis
    return Promise.race([
      (this.busy[k] || this.fetch(text, sl, tl)).then((t) => t || text),
      new Promise((res) => setTimeout(() => res(this.cache[k] || text), ms)),
    ]);
  },
  fetch(text, sl, tl) {
    const k = tl + '|' + text;
    if (this.busy[k]) return this.busy[k];
    const p = translateOnline(text, sl, tl).then((t) => {
      if (t && t !== text.trim()) {
        this.cache[k] = t;
        const ks = Object.keys(this.cache);
        if (ks.length > 150) ks.slice(0, ks.length - 150).forEach((x) => delete t
        store.set('mt', this.cache);
        scheduleRender();
      }
      return this.cache[k] || null;
    }).catch(() => null);
    this.busy[k] = p;
    p.then(() => setTimeout(() => { if (!this.cache[k]) delete this.busy[k]; }, 3
    return p;
  },
};

/* ==================== language picker ==================== */
const TICK = '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-w
function openLang() {
  $('#langList').innerHTML = I18N.langs.map((l) => {
    const here = T(l.ar);                                          // the language
    return `<button class="lang-opt${l.code === I18N.lang ? ' on' : ''}" onclick=
      <b lang="${l.code}" dir="${l.dir}" style="font-family:${l.label}">${l.name}
  }).join('');
  show('langSheet');
}
function pickLang(code) {
  closeSheets();
  if (code === I18N.lang) return;
  setTimeout(() => {
    const app = $('#app');
```

```
    app.animate && app.animate([{ opacity: 1 }, { opacity: 0.35 }, { opacity: 1 }
    setTimeout(() => I18N.set(code), 140);
  }, 260);
}
// everything drawn by app.js is redrawn in the new language (static text is hand
document.addEventListener('langchange', () => {
  updatePill();
  $('#pageName').textContent = T(PAGE_NAMES[current] || '');
  if (k3d) $('#tawafBtn span').textContent = T(k3d.tawaf ? 'تشغيل الطواف' : 'طواف
  renderContact();
  renderPlaces();
  renderMyLoc(mlState === 'ok' && !Geo.last ? 'idle' : mlState);
  if (lastFinder && $('#fResult').innerHTML) renderFinder(lastFinder, false);
  if ($('#reportSheet').classList.contains('show')) { renderReportFor(); setLocRo
  if ($('#alarm').classList.contains('show') && S.reports[alarmRid]) fillAlarm(S.
  renderConn();
  lastReportsHTML = ''; notifSig = '';
  if (Session.role === 'super') renderGoogle(true);
  scheduleRender();
  if (current === 'home') runRoute();
  if (map) setTimeout(() => map.invalidateSize(), 60);
});

/* ===================== start ===================== */
(function boot() {
  Session.load();
  loadCache();
  if (Session.role) {
    Relay.connect();
    if (Session.role === 'pilgrim') { const r = myOpenReport(); if (r) startLiveS
    if (Session.role === 'super') Geo.watch(onSupFix);
  }
  updatePill();
  renderHome();
  runRoute();
  scheduleRender();
  new IntersectionObserver((es) => { heroVisible = es[0].isIntersecting; updateK3
  requestAnimationFrame(() => setTimeout(initK3D, 60));
  if (isIOS() && !isStandalone()) $('#installCard').classList.remove('hidden');
  if ('serviceWorker' in navigator && /^https?:/.test(location.protocol)) navigat
  if (location.hash === '#super') setTimeout(openFromNotification, 400);
  if (Session.role === 'super') Push.refresh();
})();
```